



INSTITUTO FEDERAL

Fluminense

Campus

Bom Jesus do Itabapoana

Projeto e análise de algoritmos

Medidas de Complexidade

Notação Assintótica

Análise Assintótica de Limites de Complexidade

ana.figueiredo@iff.edu.br

Algoritmos

- Como resolver um problema no computador?
 - Precisamos descrevê-lo de uma forma clara e precisa
- Precisamos escrever o seu **algoritmo**
 - Um **algoritmo** é uma sequência simples e objetiva de **instruções**
 - Cada **instrução** é uma informação que indica ao computador uma ação básica a ser executada
- Vários algoritmos para um mesmo problema
 - Os algoritmos se diferenciam uns dos outros pela maneira como eles utilizam os recursos do computador
- Os algoritmos dependem
 - Principalmente do tempo que demora para ser executado
 - Da quantidade de memória do computador

Análise de algoritmos

- Área de pesquisa cujo foco são os algoritmos
 - Busca responder a seguinte pergunta: **podemos fazer um algoritmo mais eficiente?**
 - Algoritmos diferentes mas capazes de resolver o mesmo problema não necessariamente o fazem como a mesma eficiência
 - Exemplo: ordenação de números
- As diferenças de eficiência podem ser
 - **Irrelevantes** para um pequeno número de elementos processados
 - **Crescer proporcionalmente** com o número de elementos processados
- Dependendo do tamanho dos dados e da eficiência, um programa poderia executar
 - Instantaneamente
 - De um dia para o outro
 - Por séculos

Importância Análise de algoritmos

Eficiência e Desempenho:

- A análise de algoritmos permite identificar algoritmos que executam tarefas de maneira mais eficiente, economizando tempo de processamento e recursos de hardware.
- Algoritmos eficientes são vitais para a construção de sistemas responsivos e escaláveis.

Economia de Recursos:

- Algoritmos ineficientes podem consumir recursos valiosos, como energia, memória e poder de processamento.
- A análise de algoritmos ajuda a otimizar o uso desses recursos, tornando os sistemas mais sustentáveis.

Seleção de Algoritmos Adequados:

- A análise ajuda a escolher o algoritmo certo para resolver um problema específico, considerando fatores como tamanho da entrada e restrições de tempo.
- Isso evita o uso desnecessário de algoritmos complexos quando algoritmos mais simples são adequados.

Projeto de Software de Alta Qualidade:

- Algoritmos bem projetados contribuem para a construção de software de alta qualidade, estável e confiável.
- A análise ajuda a evitar gargalos de desempenho que podem afetar a usabilidade e a satisfação do usuário.

Inovação e Otimização:

- A análise de algoritmos impulsiona a inovação ao motivar os engenheiros a buscar soluções mais rápidas e eficientes para os desafios computacionais.
- Algoritmos eficientes podem ser a base para a criação de novas tecnologias e aplicações.

Adaptação a Novas Tecnologias:

- Com a evolução da tecnologia, a análise de algoritmos permite avaliar como novos algoritmos se comportam em diferentes ambientes, como dispositivos móveis, sistemas distribuídos e computação em nuvem.

Desenvolvimento Competitivo:

- Em um mundo onde a velocidade e a eficiência são cruciais, empresas que podem oferecer produtos e serviços com melhor desempenho tendem a se destacar no mercado.

Resolução de Problemas Complexos:

- Muitos problemas do mundo real são complexos e exigem algoritmos eficientes para serem resolvidos em tempo viável.
- A análise de algoritmos capacita os engenheiros a abordar esses desafios de forma eficaz.

A análise de algoritmos é o alicerce sobre o qual a engenharia de software constrói soluções computacionais eficientes, sustentáveis e inovadoras. Ela permite que os profissionais tomem decisões informadas sobre a escolha e o design de algoritmos, contribuindo para o progresso contínuo na área de tecnologia da informação.

Análise de algoritmos

- **Complexidade computacional**

- Medida criada para comparar a eficiência dos algoritmos
- Indica o **custo** ao se aplicar um algoritmo

custo = memória + tempo

- **memória**: quanto de espaço o algoritmo vai consumir
- **tempo**: a duração de sua execução

Análise de algoritmos

- Importante
 - O custo pode estar associado a outros recursos computacionais, além da memória
 - Exemplo: tráfego de rede
 - No entanto, para a maior parte dos problemas o custo está relacionado ao tempo de execução em função do **tamanho da entrada** a ser processada

Análise de algoritmos

- Para determinar se um algoritmo é o mais eficiente, podemos utilizar duas abordagens
 - **Análise empírica**
 - Comparação entre os programas
 - **Análise matemática**
 - Estudo das propriedades do algoritmo

Ambas as abordagens têm seus méritos e são valiosas na análise de algoritmos. A análise empírica oferece uma compreensão direta do desempenho em um ambiente específico, enquanto a análise matemática proporciona insights gerais que são aplicáveis a uma variedade de cenários. Em muitos casos, é benéfico combinar ambas as abordagens para obter uma visão completa do desempenho de um algoritmo.

Aspecto	Análise Empírica	Análise Matemática
Natureza	Baseada em experimentos práticos e medições.	Baseada em fórmulas matemáticas e notações.
Abordagem	Requer execução real do algoritmo.	Usa equações para representar a complexidade.
Precisão	Pode ser afetada por variações de hardware, SO, etc.	Fornecer resultados precisos sob certas condições.
Escalabilidade	Pode ser difícil testar para todas as entradas.	Fornece uma visão abrangente para qualquer entrada.
Fatores não considerados	Detalhes de implementação, otimizações de hardware.	Considera apenas os aspectos teóricos do algoritmo.
Utilidade	Útil para medir o desempenho real em um ambiente específico.	Útil para avaliar o comportamento geral do algoritmo.
Limitações	Pode não capturar o comportamento em todas as situações.	Pode não levar em conta todas as nuances práticas.
Ferramentas	Perfis, benchmarks, ferramentas de medição de tempo.	Notação O-grande, cálculos matemáticos.

Análise empírica

- Definição

- Avalia o custo (ou complexidade) de um algoritmo a partir da avaliação da execução do mesmo quando implementado
- Análise pela execução de seu programa correspondente

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main() {
    clock_t inicio, fim;
    unsigned long int tempo;
    inicio = clock();
    //=====
    /* coloque seu código aqui */
    //=====
    fim = clock();
    tempo = (fim - inicio)*1000/CLOCKS_PER_SEC;
    printf("tempo: %lu milissegundo\n", tempo);

    return 0;
}
```

Análise empírica - vantagens

- Realismo Prático: A análise empírica leva em consideração fatores do mundo real, como variações de hardware, sistemas operacionais e cargas de trabalho específicas.
- Avaliação de Impacto: Permite avaliar o impacto de otimizações específicas, melhorias de código e mudanças de plataforma em um ambiente real.
- Comportamento Sob Condições Reais: Fornece informações sobre como um algoritmo se comporta em situações de uso reais, considerando entradas variáveis.
- Detecção de Anomalias: Ajuda a identificar casos excepcionais em que o desempenho real difere da análise teórica, permitindo correções.
- Medição de Recursos: Pode medir recursos como uso de CPU, memória e largura de banda, que podem ser críticos em sistemas com recursos limitados.

Análise empírica - desvantagens

- Dependência do Ambiente: O desempenho empírico pode variar de sistema para sistema, tornando difícil a generalização para diferentes ambientes. Dependência do programador, por exemplo.
- Custo e Tempo: A execução de experimentos práticos pode ser demorada e custosa em termos de recursos de hardware e tempo.
- Entradas Limitadas: Não é possível testar todos os possíveis cenários de entrada, o que pode resultar em conclusões limitadas.
- Reprodutibilidade: Diferenças sutis no ambiente ou nas condições de teste podem dificultar a reprodutibilidade dos resultados.
- Influência de Outros Processos: A execução em um ambiente real pode ser afetada por outros processos em execução no sistema.
- Validade Estatística: Para conclusões confiáveis, é necessário um número suficiente de experimentos repetidos.

Análise matemática

- Muitas vezes, é preferível que a medição do tempo gasto por um algoritmo seja feita de maneira independente
 - Neste caso, devemos desconsiderar o hardware ou a linguagem de programação usada, convém utilizar a **análise matemática** do algoritmo.
- Permite um estudo formal de um algoritmo ao nível da sua idéia
 - Faz uso de um computador idealizado e simplificações que buscam considerar somente os custos dominantes do algoritmo
 - Detalhes de baixo nível são ignorados
 - Linguagem de programação utilizada
 - Hardware no qual o algoritmo é executado
 - Conjunto de instruções da CPU
 - Etc

Análise matemática - vantagens

- **Precisão Teórica:** A análise matemática permite uma compreensão precisa e abstrata do comportamento dos algoritmos. Ela fornece limites e estimativas exatas para o desempenho.
- **Generalização:** Uma análise matemática bem-feita fornece informações valiosas sobre o comportamento do algoritmo para todas as entradas possíveis, não apenas para conjuntos específicos de testes.
- **Comparação Objetiva:** Através da análise matemática, é possível comparar algoritmos diretamente e determinar qual deles é mais eficiente em termos de tempo e espaço, independentemente do ambiente de execução.
- **Previsibilidade:** A análise matemática permite prever como o algoritmo se comporta à medida que o tamanho da entrada aumenta, o que é essencial para projetar sistemas escaláveis.

Análise matemática - desvantagens

- **Simplificações Excessivas:** Às vezes, a análise matemática pode envolver simplificações que não refletem completamente o mundo real. Algoritmos que são eficientes na teoria podem não ser tão eficientes em implementações práticas.
- **Não Considera Detalhes Práticos:** A análise matemática muitas vezes ignora detalhes de implementação, otimizações de hardware e outros fatores do mundo real, que podem afetar significativamente o desempenho.
- **Comportamento Assintótico:** A análise matemática tende a se concentrar no comportamento assintótico, ou seja, à medida que o tamanho da entrada tende ao infinito. Isso pode não refletir o comportamento em escalas menores.
- **Dependência das Hipóteses:** Muitas análises matemáticas baseiam-se em hipóteses específicas sobre a entrada ou o ambiente de execução, que podem não ser sempre verdadeiras na prática.
- **Complexidade:** Algumas análises matemáticas podem se tornar extremamente complexas, especialmente para algoritmos mais sofisticados, tornando-as difíceis de serem compreendidas e aplicadas corretamente.

Contando instruções

- Considere o pequeno trecho de código
 - Este algoritmo procura o maior valor presente em um array **A** contendo **n** elementos e o armazena na variável **M**

```
int M = A[0];

for(i = 0; i < n; i++) {
    if(A[i] >= M) {
        M = A[i];
    }
}
```

Contando instruções

- Quantas **instruções simples** ele executa?
 - **Instruções simples**: instruções que podem ser executadas diretamente pelo CPU (ou algo muito perto disso)
 - atribuição de um valor a uma variável
 - acesso ao valor de um determinado elemento do array
 - comparação de dois valores
 - incremento de um valor
 - operações aritméticas básicas, como adição e multiplicação

Contando instruções

- Instruções simples

- Todas possuem o mesmo custo
- Comandos de seleção (como o comando **if**) possuem custo zero
 - Não contam como instruções

```
int M = A[0];  
  
for(i = 0; i < n; i++) {  
    if(A[i] >= M) {  
        M = A[i];  
    }  
}
```

Contando instruções

- Custo da inicialização de **M**: 1 instrução
 - Apenas uma operação de atribuição

```
int M = A[0];  
  
for(i = 0; i < n; i++) {  
    if(A[i] >= M) {  
        M = A[i];  
    }  
}
```

Contando instruções

- Custo de inicialização do laço **for**: 2 instruções
 - Uma operação de atribuição e uma comparação

```
int M = A[0];  
for(i = 0; i < n; i++) {  
    if(A[i] >= M) {  
        M = A[i];  
    }  
}
```

Contando instruções

- Custo de execução do laço **for**: $2n$ instruções
 - Uma operação de incremento e uma comparação executadas n vezes

```
int M = A[0];  
  
for(i = 0; i < n; i++) {  
    if(A[i] >= M) {  
        M = A[i];  
    }  
}
```

Contando instruções

- Ignorando os comandos contidos dentro do laço **for**, temos que o algoritmo irá executar **$3+2n$** instruções
 - 3 instruções antes de iniciar o laço **for**
 - 2 instruções ao final de cada laço **for**

```
int M = A[0];  
for(i = 0; i < n; i++){  
    if(A[i] >= M) {  
        M = A[i];  
    }  
}
```

Contando instruções

- Assim, considerando um **laço vazio**, podemos definir uma função matemática que representa o custo do algoritmo em relação ao tamanho do array de entrada
 - $f(n) = 2n + 3$

```
int M = A[0];  
for(i = 0; i < n; i++){  
    if(A[i] >= M) {  
        M = A[i];  
    }  
}
```


Contando instruções

- As instruções vistas anteriormente eram sempre executadas. Porém, as instruções dentro do **for** podem ou não ser executadas
 - Comando de seleção: 1 instrução
 - Sempre executada
 - Atribuição: 1 instrução
 - Depende do resultado do comando de seleção

```
int M = A[0];  
  
for (i = 0; i < n; i++) {  
    if (A[i] >= M) {  
        M = A[i];  
    }  
}
```

Contando instruções

- Antes, bastava saber o tamanho do array, n , para definir a função de custo $f(n)$
- Agora, temos que considerar também o conteúdo do array
- Tome como exemplo os dois arrays abaixo

```
int A1[4] = {1, 2, 3, 4};  
int A2[4] = {4, 3, 2, 1};
```

Contando instruções

- O array **A1** irá executar mais instruções do que o array **A2**
 - Array **A1**: o comando **if** é sempre **verdadeiro**
 - Array **A2**: o comando **if** é sempre **falso**
- Devemos considerar o pior caso possível
 - Maior número de instruções é executado

```
int A1[4] = {1, 2, 3, 4};  
int A2[4] = {4, 3, 2, 1};
```

Contando instruções

- Neste exemplo, o **pior caso** ocorre quando o array possui valores em ordem crescente
 - Valor de **M** é sempre substituído: $2n$ instruções
 - Maior número de instruções
 - A função custo será, no **pior caso**,
 - $f(n) = 3 + 2n + 2n$ ou
 - $f(n) = 4n + 3$

```
int M = A[0];  
  
for (i = 0; i < n; i++) {  
    if (A[i] >= M) {  
        M = A[i];  
    }  
}
```

Contando instruções

- Exemplo: maior valor presente em uma matriz **A** contendo **n x n** elementos

```
M = A[0][0];  
  
for(i=0; i<n; i++)  
    for(j=0; j<n; j++)  
        if(M < A[i][j])  
            M = A[i][j];
```

$$f(n) = 1 + 2 + 2n + n(2 + 2n + 2n)$$

$$f(n) = 3 + 2n + 2n + 2n^2 + 2n^2$$

$$f(n) = 3 + 4n + 4n^2$$

Comportamento assintótico

- Essa é a função de complexidade de tempo
 - Nos dá uma ideia do custo de execução do algoritmo para um problema de tamanho **n**
 - Exemplo: array de **n** elementos
 - É possível criar o mesmo tipo de função para a análise do espaço gasto

```
int M = A[0];

for(i = 0; i < n; i++) {
    if(A[i] >= M) {
        M = A[i];
    }
}
```

Comportamento assintótico

- Dúvida
 - Será que todos os termos da função $f(n)$ são necessários para termos uma noção do custo?
- De fato, nem todos os termos são necessários
 - Podemos descartar certos termos na função
 - Devemos manter apenas os que nos dizem o que acontece quando o tamanho dos dados de entrada (n) cresce muito

Comportamento assintótico

- Idéia geral
 - Se um algoritmo é mais rápido do que outro para um grande conjunto de dados de entrada, é muito provável que ele continue sendo também mais rápido em um conjunto de dados menor
 - Assim, podemos
 - Descartar todos os termos que crescem lentamente
 - Manter apenas os que crescem mais rápido à medida que o valor de n se torna maior

Comportamento assintótico

- A função $f(n) = 4n + 3$ possui dois termos
 - $4n$ e 3
- O termo 3 é uma **constante de inicialização**
 - Não se altera à medida que o valor de n cresce
 - Exemplo: atribuições antes de um laço
 - Pode, portanto, ser descartado
- Assim, a função é reduzida para $f(n) = 4n$

Comportamento assintótico

- Constantes que multiplicam o termo n da função também devem ser descartadas
 - Representam particularidades de cada linguagem e compilador
 - Queremos analisar apenas a ideia por trás do algoritmo, sem influências da linguagem

Comportamento assintótico

- Exemplo: atribuição em array
 - Na linguagem Pascal, essa operação equivale a um teste lógico e uma atribuição na linguagem C
 - Pascal: 3 instruções (etapa de verificação)
 - C: 1 instrução (sem etapa de verificação)

```
//Pascal
M:= A[i];
//C
if(i >=0 && i < n)
    M = A[i];
```

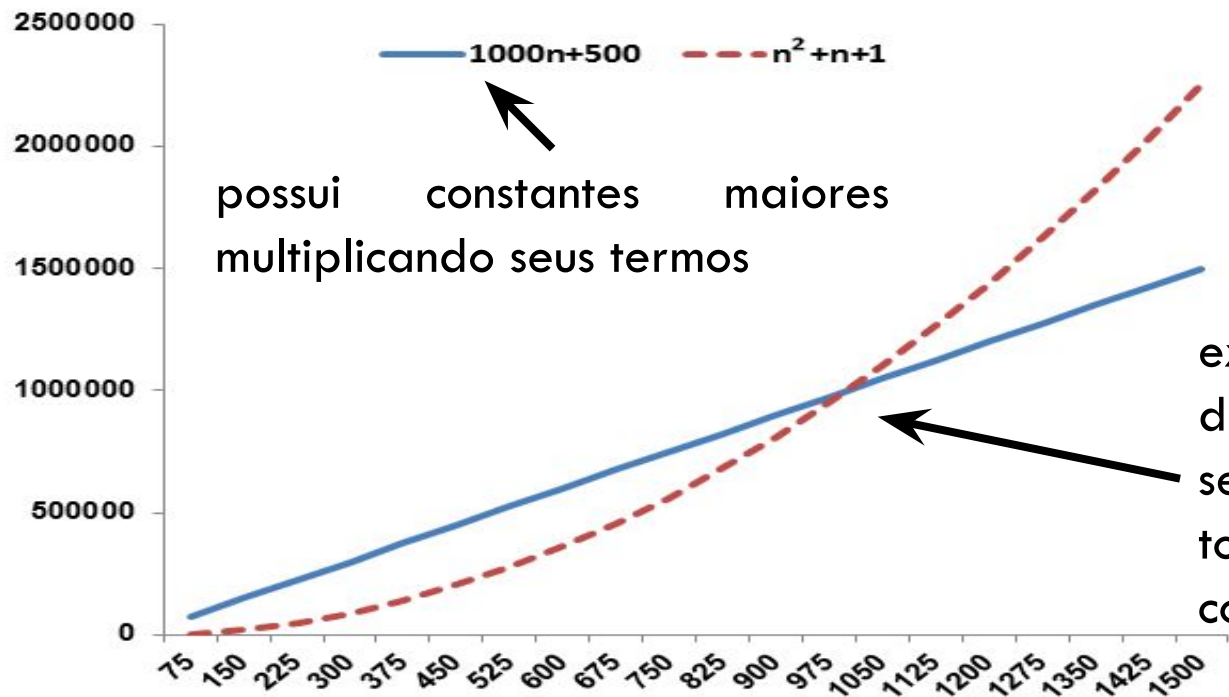
- Assim, a função $f(n) = 4n + 3$ é reduzida para $f(n) = n$

Comportamento assintótico

- Descartando todos os termos constantes e mantendo apenas o de maior crescimento obtemos o **comportamento assintótico**
 - Comportamento de uma função $f(n)$ quando n tende ao infinito
 - O termo de maior expoente domina o comportamento da função

Comportamento assintótico

- Exemplo: $g(n) = 1000n+500$ e $h(n) = n^2+n+1$



possui constantes maiores multiplicando seus termos

existe um valor de n a partir do qual o resultado de $h(n)$ é sempre maior do que $g(n)$, tornando os demais termos e constantes pouco importantes

Comportamento assintótico

- Assim, podemos
 - Suprimir os termos menos importantes da função e considerar apenas o termo de **maior grau**
 - Descrever a complexidade usando somente o seu custo dominante
 - n para a função $g(n)$
 - n^2 para $h(n)$

Comportamento assintótico

- Exemplos de função de custo juntamente com o seu comportamento assintótico
 - **Obs:** Se a função não possui nenhum termo multiplicado por **n**, seu comportamento assintótico é constante

Função custo	Comportamento assintótico
$f(n) = 105$	$f(n) = 1$
$f(n) = 15n + 2$	$f(n) = n$
$f(n) = n^2 + 5n + 2$	$f(n) = n^2$
$f(n) = 5n^3 + 200n^2 + 112$	$f(n) = n^3$

Comportamento assintótico

- De modo geral, podemos obter a função de custo de um programa simples apenas contando os comandos de laços aninhados
 - Não possui laço (exceto se houver recursão)
 - $f(n) = 1$
 - Um comando de laço indo de 1 a n
 - $f(n) = n$
 - Dois comandos de laço aninhados
 - $f(n) = n^2$
 - e assim por diante

O comportamento assintótico é um conceito importante na análise de algoritmos que se concentra no desempenho dos algoritmos à medida que o tamanho da entrada se torna muito grande. Ele ajuda a entender como um algoritmo se comporta quando a entrada cresce em direção ao infinito, ignorando fatores constantes e termos de menor ordem que podem não ser tão significativos em entradas grandes

Diferentes tipos de análise assintótica

Notação Ω (Omega): A notação Ω é usada para descrever um limite inferior assintótico de uma função. Ou seja, ela fornece um limite inferior para o crescimento da função, indicando que a função não cresce mais devagar do que a função especificada pela notação Ω . Formalmente, $f(n) = \Omega(g(n))$ significa que existe uma constante positiva c e um tamanho de entrada mínimo n_0 tal que $f(n) \geq c * g(n)$ para todos os valores de n maiores ou iguais a n_0 .

Notação ω (Small Omega): A notação ω é usada para descrever limites inferiores assintóticos que não são apertados. Ela indica que a função cresce mais rapidamente do que a função especificada. Formalmente, $f(n) = \omega(g(n))$ significa que para qualquer constante positiva c , não importa quão pequena, existe um tamanho de entrada mínimo n_0 tal que $f(n) > c * g(n)$ para todos os valores de n maiores ou iguais a n_0 .

Diferentes tipos de análise assintótica

Notação O (Big O): A notação O é usada para descrever um limite superior assintótico de uma função. Ela fornece um limite superior para o crescimento da função, indicando que a função não cresce mais rapidamente do que a função especificada pela notação O. Formalmente, $f(n) = O(g(n))$ significa que existe uma constante positiva c e um tamanho de entrada mínimo n_0 tal que $f(n) \leq c * g(n)$ para todos os valores de n maiores ou iguais a n_0 .

Notação o (Small O): A notação o é usada para descrever limites superiores assintóticos que não são apertados. Ela indica que a função cresce mais lentamente do que a função especificada. Formalmente, $f(n) = o(g(n))$ significa que para qualquer constante positiva c , não importa quão pequena, existe um tamanho de entrada mínimo n_0 tal que $f(n) < c * g(n)$ para todos os valores de n maiores ou iguais a n_0 .

Notação Θ (Theta): A notação Θ é usada para descrever um limite apertado ou limites superior e inferior iguais para o crescimento de uma função. Ela indica que a função está limitada tanto superior quanto inferiormente pela mesma função, dentro de um certo intervalo. Formalmente, $f(n) = \Theta(g(n))$ significa que existem constantes positivas c_1 , c_2 e um tamanho de entrada mínimo n_0 tal que $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ para todos os valores de n maiores ou iguais a n_0 .

Notação Grande-O

- Das várias formas de análise assintótica, a mais conhecida e utilizada é a notação **grande-O** (O)
 - Custo do algoritmo no **pior caso** possível para todas as entradas de tamanho n
 - Analisa o limite superior de entrada
 - Permite dizer que o comportamento do nosso algoritmo não pode nunca ultrapassar um certo limite

Notação Grande-O

- Para entender essa notação, considere o algoritmo de ordenação **selection sort**
 - Dado um array **V** de tamanho **n**, procure o menor valor (posição **me**) e coloque na primeira posição
 - Repetir processo para a segunda posição, depois para a terceira etc.
 - Pare quando o array estiver ordenado

```
void selectionSort(int *V, int n){
    int i, j, me, troca;
    for(i = 0; i < n-1; i++){
        me = i;
        for(j = i+1; j < n; j++){
            if(V[j] < V[me])
                me = j;
        }
        if(i != me){
            troca = V[i];
            V[i] = V[me];
            V[me] = troca;
        }
    }
}
```

Notação Grande-O

- Dois comandos de laço no **selection sort**
 - Laço externo: executado **n-1 vezes**
 - Laço interno: número de execuções depende do valor do índice do laço externo (n-1, n-2, n-3, ..., 2, 1)

Procura o menor valor.

Número de execuções
depende do laço externo
n-1 vezes na 1ª iteração
n-2 vezes na 2ª iteração
etc

```
void selectionSort(int *V, int n){  
    int i, j, me, troca;  
    for(i = 0; i < n-1; i++){  
        me = i;  
        for(j = i+1; j < n; j++){  
            if(V[j] < V[me])  
                me = j;  
        }  
        if(i != me){  
            troca = V[i];  
            V[i] = V[me];  
            V[me] = troca;  
        }  
    }  
}
```

Repete o processo para
cada posição do array
Executado **n-1** vezes

Notação Grande-O

- Como calcular o custo do **selection sort**?
 - Temos que calcular o resultado da soma
 - $1 + 2 + \dots + (n - 1) + n$
 - Essa soma representa o número de execuções do laço interno
 - Dependendo do algoritmo, isso pode ser uma tarefa muito complicada
- Neste caso, temos uma ajuda da matemática
 - A soma $1 + 2 + \dots + (n - 1) + n$ equivale a soma dos n termos de uma progressão aritmética, S_n , de razão 1, Assim
 - $S_n = 1 + 2 + \dots + (n - 1) + n$
 - $S_n = n + (n - 1) + \dots + 1 + 1$
 - $2S_n = (1 + n) + (2 + (n - 1)) + \dots + ((n - 1) + 2) + (n + 1)$

Notação Grande-O

- Vamos analisar os termos equidistantes dos extremos
 - Como 1 e n, 2 e (n-1), ...,
 - Suas somas são sempre iguais a (1 + n)
 - Logo
 - $2S_n = (1 + n) + (2 + (n-1)) + \dots + ((n - 1) + 2) + (n + 1)$
 - $2S_n = n(1 + n)$
 - $S_n = n(1 + n)/2$

Notação Grande-O

- Como resultado, o número de execuções do laço interno é $S_n = n(1 + n)/2$
 - Sabemos agora o número de execuções do laço interno
 - Porém, essa é uma tarefa trabalhosa!

Notação Grande-O

- Uma alternativa mais simples: **estimar um limite superior**
 - Podemos alterar mentalmente o algoritmo e, em seguida, calcular o custo desse novo algoritmo
 - Desse modo, vamos torná-lo **menos eficiente**
 - Assim, saberemos que o algoritmo original é no máximo tão ruim, ou talvez melhor, que o novo algoritmo

Notação Grande-O

- Como diminuir a eficiência do **selection sort**?

- Podemos trocar o laço interno por um laço que seja executado sempre **n** vezes
 - Fica mais fácil descobrir o custo do algoritmo
 - Piora o desempenho: algumas execuções do laço interno serão inúteis

```
void selectionSort(int *V, int n){  
    int i, j, me, troca;  
    for(i = 0; i < n-1; i++){  
        me = i;  
        for(j = i+1; j < n; j++){  
            if(V[j] < V[me])  
                me = j;  
        }  
        if(i != me){  
            troca = V[i];  
            V[i] = V[me];  
            V[me] = troca;  
        }  
    }  
}
```

Notação Grande-O

- Temos agora dois comandos de laço aninhados sendo executados **n** vezes cada
 - Função de custo passa a ser **$f(n) = n^2$**
 - Utilizando a notação **grande-O**, **O**, podemos dizer que o custo do algoritmo no **pior caso** é **$O(n^2)$**

Notação Grande-O

- O que **$O(n^2)$** significa para um algoritmo?
 - A notação **$O(n^2)$** nos diz que o custo do algoritmo não é, assintoticamente, pior do que **n^2**
 - Nosso algoritmo nunca vai ser mais lento do que um determinado limite
 - Ou seja, o custo do algoritmo original é no máximo tão ruim quanto **n^2**
 - Pode ser melhor, mas nunca pior
 - **Limite superior** para a complexidade real do algoritmo

Diferentes tipos de análise assintótica

- A notação grande-O é a forma mais conhecida e utilizada de análise assintótica
 - Complexidade do nosso algoritmo no pior caso
 - Seja de tempo ou de espaço
 - É o caso mais fácil de se identificar
 - Limite superior sobre o tempo de execução do algoritmo
 - Para diversos algoritmos o pior caso ocorre com frequência

Diferentes tipos de análise assintótica

- Notação grande-Omega, Ω
 - Descreve o **limite assintótico inferior**
 - É utilizada para analisar o **melhor caso** do algoritmo
 - A notação $\Omega(n^2)$ nos diz que o custo do algoritmo é, assintoticamente, maior ou igual a n^2
 - Ou seja, o custo do algoritmo original é no mínimo tão ruim quanto n^2

Diferentes tipos de análise assintótica

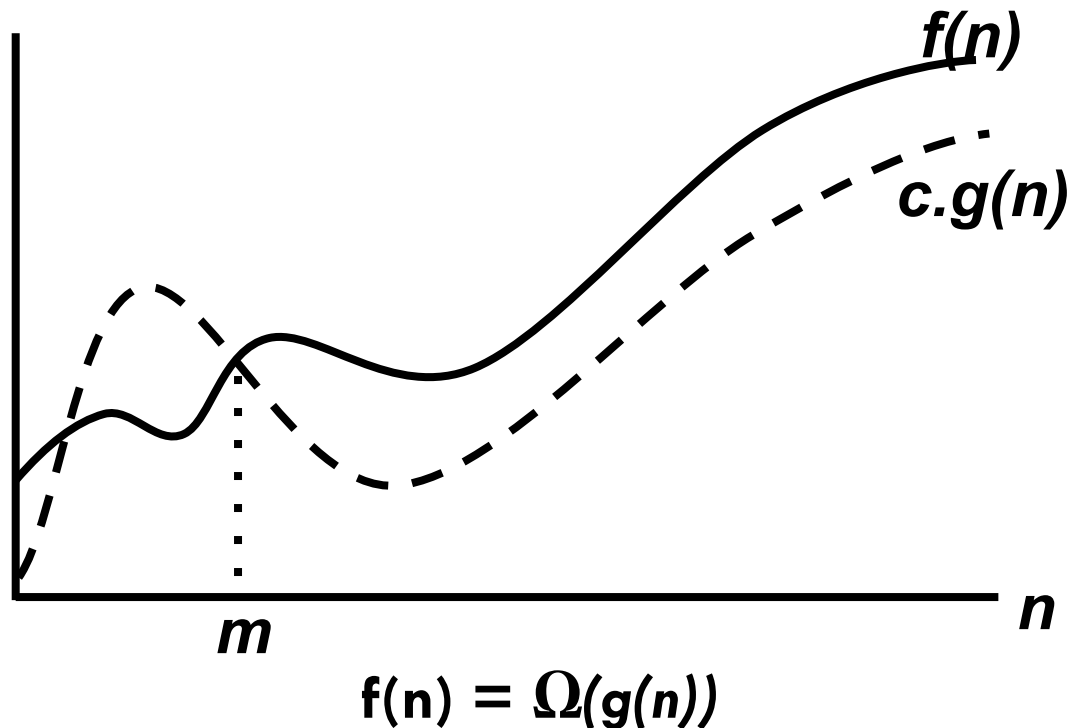
- Notação grande-Omega, Ω
 - Matematicamente, a notação Ω é assim definida
 - Uma função custo $f(n)$ é $\Omega(g(n))$ se existem duas constantes positivas c e m tais que
 - Para $n \geq m$, temos $f(n) \geq c \cdot g(n)$
 - Confuso?

Diferentes tipos de análise assintótica

- Notação grande-Omega, Ω
 - Em outras palavras, para todos os valores de n à direita de m , o resultado da função custo $f(n)$ é sempre **maior ou igual** ao valor da função usada na notação Ω , $g(n)$, multiplicada por uma constante c

Diferentes tipos de análise assintótica

- Notação grande-Omega, Ω



Diferentes tipos de análise assintótica

- Exemplo: mostrar que a função custo é **$f(n)=3n^2 + n$ é $\Omega(n)$**
 - Temos que encontrar constantes **c** e **m** tais que
 - $3n^2 + n \geq cn$
 - Dividindo por n^2 , temos
 - $3 + 1/n \geq c/n$
 - Considerando **$c = 4$** e **$n > 0$** , temos que **$f(n)=3n^2+n$ é $\Omega(n)$**

Diferentes tipos de análise assintótica

- Notação grande-O, O
 - Descreve o **limite assintótico superior**
 - É utilizada para analisar o **pior caso** do algoritmo
 - A notação $O(n^2)$ nos diz que o custo do algoritmo é, assintoticamente, menor ou igual a n^2
 - Ou seja, o custo do algoritmo original é no máximo tão ruim quanto n^2

Diferentes tipos de análise assintótica

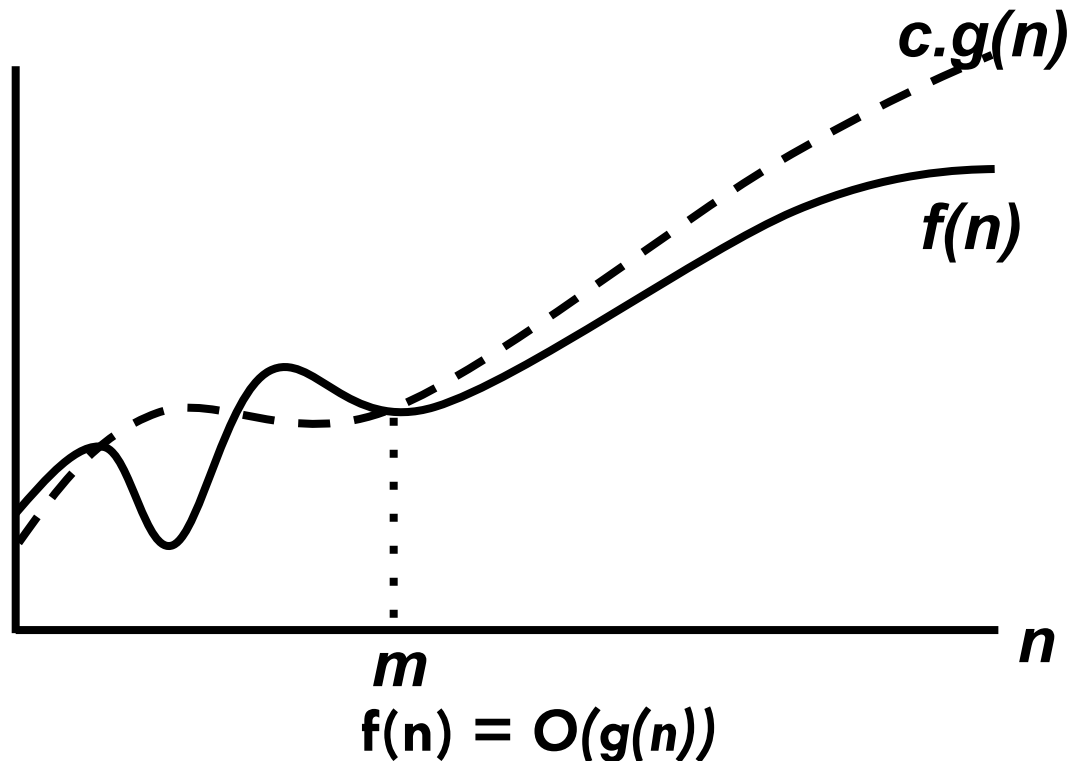
- Notação grande-O, O
 - Matematicamente, a notação O é assim definida
 - Uma função custo $f(n)$ é $O(g(n))$ se existem duas constantes positivas c e m tais que
 - Para $n \geq m$, temos $f(n) \leq c \cdot g(n)$
 - Confuso?

Diferentes tipos de análise assintótica

- Notação grande-O, O
 - Em outras palavras, para todos os valores de n à direita de m , o resultado da função custo $f(n)$ é sempre **menor ou igual** ao valor da função usada na notação O , $g(n)$, multiplicada por uma constante c .

Diferentes tipos de análise assintótica

- Notação grande-O, O



Diferentes tipos de análise assintótica

- Exemplo: mostrar que a função custo é **$f(n)=2n^2 + 10$ é $O(n^3)$**
 - ▣ Temos que encontrar constantes **c** e **m** tais que
 - ▣ $2n^2 + 10 \leq cn^3$
 - ▣ Dividindo por n^2 , temos
 - ▣ $2 + 10/n^2 \leq cn$
 - ▣ Considerando **$c = 1$** e **$n > 3$** , temos que **$f(n)=2n^2+10$ é $O(n^3)$**
 - Dá para melhorar essa análise!

Diferentes tipos de análise assintótica

- Exemplo: mostrar que a função custo é **$f(n)=2n^2 + 10$ é $O(n^2)$**
 - Temos que encontrar constantes **c** e **m** tais que
 - $2n^2 + 10 \leq cn^2$
 - Dividindo por **n^2** , temos
 - $2 + 10/n^2 \leq c$
 - Considerando **$c = 12$** e **$n > 0$** , temos que **$f(n)=2n^2+10$ é $O(n^2)$**

Diferentes tipos de análise assintótica

- Exemplo: mostrar que a função custo é **$f(n)=4n + 7$ é $O(n)$**
 - ▣ Temos que encontrar constantes **c** e **m** tais que
 - ▣ $4n + 7 \leq cn$
 - ▣ Dividindo por **n** , temos
 - ▣ $4 + 7/n \leq c$
 - ▣ Considerando **$c = 8$** e **$n > 1$** , temos que **$f(n)=4n+7$ é $O(n)$**

Diferentes tipos de análise assintótica

- Exemplo: mostrar que a função custo é **$f(n)=n^2$ não é $O(n)$**
 - ▣ Temos que encontrar constantes **c** e **m** tais que
 - ▣ $n^2 \leq cn$
 - ▣ Dividindo por **n** , temos
 - ▣ $n \leq c$
- ▣ A desigualdade é inválida!
 - ▣ O valor de **n** está limitado pela constante **c**
 - ▣ A análise assintótica não é possível (entrada tendendo ao infinito)

Diferentes tipos de análise assintótica

- Notação grande-O, O
 - Essa notação possui algumas operações
 - A mais importante é a **regra da soma**
 - Permite a análise da complexidade de diferentes algoritmos em sequência
 - Definição
 - Se dois algoritmos são executados em sequência, a complexidade será dada pela complexidade do maior deles
 - $O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$

Diferentes tipos de análise assintótica

- Notação grande-O, O
 - Exemplo da **regra da soma**. Se temos
 - Dois algoritmos cujos tempos de execução são $O(n)$ e $O(n^2)$, a execução deles em sequência será $O(\max(n, n^2))$ que é $O(n^2)$
 - Dois algoritmos cujos tempos de execução são $O(n)$ e $O(n \log n)$, a execução deles em sequência será $O(\max(n, n \log n))$ que é $O(n \log n)$

Diferentes tipos de análise assintótica

- Notação grande-Theta, Θ
 - Descreve o **limite assintótico firme**
 - É utilizada para analisar o **limite inferior** e **superior** do algoritmo
 - A notação $\Theta(n^2)$ nos diz que o custo do algoritmo é, assintoticamente, igual a n^2
 - Ou seja, o custo do algoritmo original é n^2 dentro de um fator constante **acima** e **abaixo**

Diferentes tipos de análise assintótica

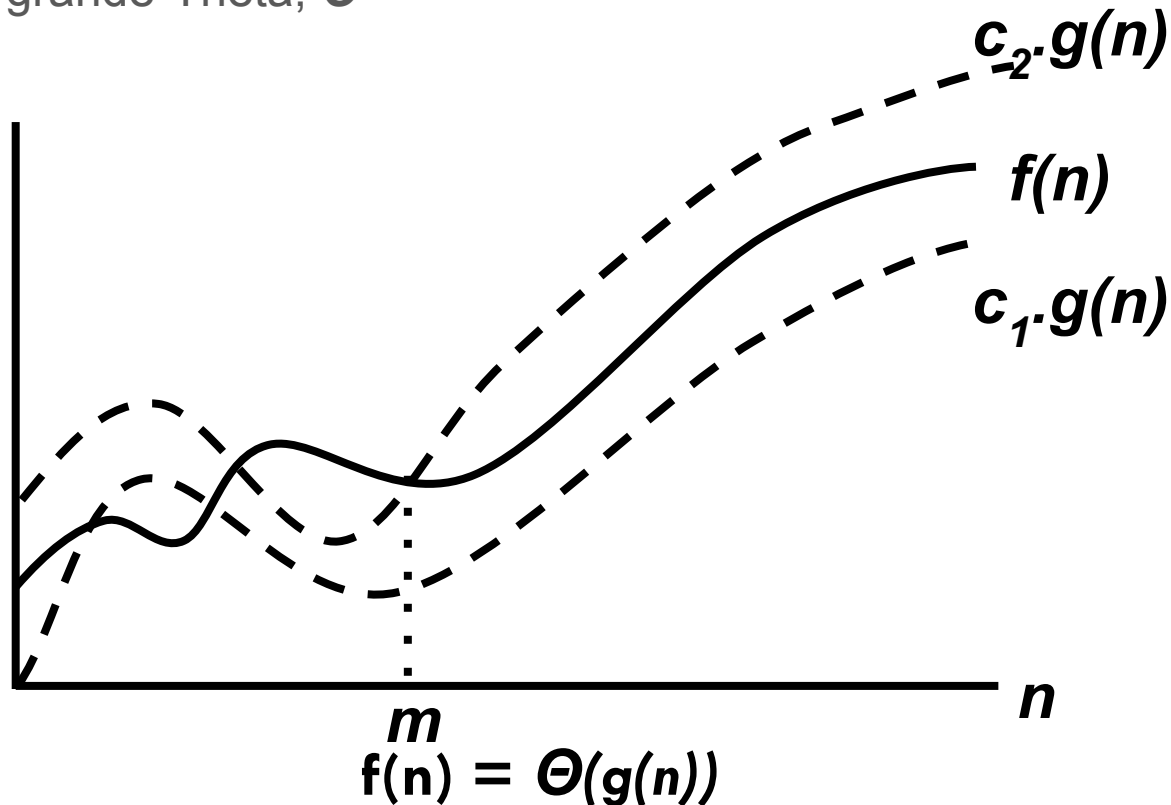
- Notação grande-Theta, Θ
 - Matematicamente, a notação Θ é assim definida
 - Uma função custo $f(n)$ é $\Theta(g(n))$ se existem três constantes positivas c_1 , c_2 e m tais que
 - Para $n \geq m$, temos $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$
 - Confuso?

Diferentes tipos de análise assintótica

- Notação grande-Theta, Θ
 - Em outras palavras, para todos os valores de n à direita de m , o resultado da função custo $f(n)$ é sempre **igual** ao valor da função usada na notação Θ , $g(n)$, quando está é multiplicada por constantes c_1 e c_2

Diferentes tipos de análise assintótica

- Notação grande-Theta, Θ



Diferentes tipos de análise assintótica

- Exemplo: mostrar que a função custo

$$f(n) = \frac{1}{2}n^2 - 3n \text{ é } \boldsymbol{\Theta(n^2)}$$

- ▣ Temos que encontrar constantes c_1 e c_2 e n tais que
- ▣ $c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$
- ▣ Dividindo por n^2 , temos
- ▣ $c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$

Diferentes tipos de análise assintótica

- Exemplo: mostrar que a função custo

$$f(n) = \frac{1}{2}n^2 - 3n \text{ é } \Theta(n^2)$$

- $c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$

- A desigualdade do lado direito é válida para $n \geq 1$ escolhendo $c_2 \geq 1/2$

- A desigualdade do lado esquerdo é válida para $n \geq 7$ escolhendo $c_1 \geq 1/14$

- Assim, para $c_1 \geq 1/14$, $c_2 \geq 1/2$ e $n \geq 7$, $f(n) = \frac{1}{2}n^2 - 3n$ é $\Theta(n^2)$

Diferentes tipos de análise assintótica

- Exemplo: mostrar que a função custo $f(n) = 6n^3$ **não é $\Theta(n^2)$**
 - ▣ Temos que encontrar constantes c_1 e c_2 e m tais que
 - ▣ $c_1 n^2 \leq 6n^3 \leq c_2 n^2$
 - ▣ Dividindo por n^2 , temos
 - ▣ $c_1 \leq 6n \leq c_2$

Diferentes tipos de análise assintótica

- Exemplo: mostrar que a função custo $f(n) = 6n^3$ **não é $\Theta(n^2)$**
 - ▣ $c_1 \leq 6n \leq c_2$
- ▣ A desigualdade do lado direito é inválida!
 - ▣ $n \leq \frac{c_2}{6}$
 - ▣ O valor de **n** está limitado pela constante **c₂**
 - ▣ A análise assintótica não é possível (entrada tendendo ao infinito)

Diferentes tipos de análise assintótica

- Notação pequeno-o, o , e pequeno-omega, ω
 - Parecidas com as notações **Grande-O** e **Grande-Omega**
 - As notações **Grande-O** e **Grande-Omega** possuem uma relação de **menor ou igual** e **maior ou igual**
 - As notações **Pequeno-o** e **Pequeno-omega** possuem uma relação de **menor** e **maior**

Diferentes tipos de análise assintótica

- Notação pequeno-o, o , e pequeno-omega, ω
 - Ou seja, essas notações não representam limites próximos da função
 - Elas representam limites estritamente
 - **superiores**: sempre maior
 - **inferiores**: sempre menor

Notação	Símbolo	Significado	Exemplo
Ω (Omega)	$f(n) \in \Omega(g(n))$	$f(n)$ cresce pelo menos tão rápido quanto $g(n)$ para entradas grandes.	$f(n) = \Omega(g(n))$
O (Big O)	$f(n) \in O(g(n))$	$f(n)$ cresce no máximo tão rápido quanto $g(n)$ para entradas grandes.	$f(n) = O(g(n))$
Θ (Theta)	$f(n) \in \Theta(g(n))$	$f(n)$ cresce tão rápido quanto $g(n)$ para entradas grandes.	$f(n) = \Theta(g(n))$
o (Little o)	$f(n) \in o(g(n))$	$f(n)$ cresce mais devagar do que $g(n)$ para entradas grandes.	$f(n) = o(g(n))$
ω (Little omega)	$f(n) \in \omega(g(n))$	$f(n)$ cresce mais rapidamente do que $g(n)$ para entradas grandes.	$f(n) = \omega(g(n))$

Classes de problemas

- A seguir, são apresentadas algumas classes de complexidade de problemas comumente usadas
 - $O(1)$: ordem constante
 - As instruções são executadas um número fixo de vezes. Não depende do tamanho dos dados de entrada
 - $O(\log n)$: ordem logarítmica
 - Típica de algoritmos que resolvem um problema transformando-o em problemas menores
 - $O(n)$: ordem linear
 - Em geral, uma certa quantidade de operações é realizada sobre cada um dos elementos de entrada

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Classes de problemas

- Mais classes de problemas
 - $O(n \log n)$: ordem log linear
 - Típica de algoritmos que trabalham com particionamento dos dados. Esses algoritmos resolvem um problema transformando-o em problemas menores, que são resolvidos de forma independente e depois unidos
 - $O(n^2)$: ordem quadrática
 - Normalmente ocorre quando os dados são processados aos pares. Uma característica deste tipo de algoritmos é a presença de um aninhamento de dois comandos de repetição

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Classes de problemas

- Mais classes de problemas
 - $O(n^3)$: ordem cúbica
 - É caracterizado pela presença de três estruturas de repetição aninhadas
 - $O(2^n)$: ordem exponencial
 - Geralmente ocorre quando se usa uma solução de **força bruta**. Não são úteis do ponto de vista prático
 - $O(n!)$: ordem fatorial
 - Geralmente ocorre quando se usa uma solução de **força bruta**. Não são úteis do ponto de vista prático. Possui um comportamento muito pior que o exponencial

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Classes de problemas

- Comparação no tempo de execução
 - Computador executa 1 milhão de operações por segundo

$f(n)$	$n = 10$	$n = 20$	$n = 30$	$n = 50$	$n = 100$
n	1,0E-05 segundos	2,0E-05 segundos	4,0E-05 segundos	5,0E-05 segundos	6,0E-05 segundos
$n \log n$	3,3E-05 segundos	8,6E-05 segundos	2,1E-04 segundos	2,8E-04 segundos	3,5E-04 segundos
n^2	1,0E-04 segundos	4,0E-04 segundos	1,6E-03 segundos	2,5E-03 segundos	3,6E-03 segundos
n^3	1,0E-03 segundos	8,0E-03 segundos	6,4E-02 segundos	0,13 segundos	0,22 segundos
2^n	1,0E-03 segundos	1,0 segundo	2,8 dias	35,7 anos	365,6 séculos
3^n	5,9E-02 segundos	58,1 minutos	3855,2 séculos	2,3E+08 séculos	1,3E+13 séculos

Hierarquia de complexidade

$O(1)$: Tempo de execução constante. O algoritmo leva o mesmo tempo, independentemente do tamanho da entrada.

Exemplo: Acesso direto a um elemento em um array. A complexidade não depende do tamanho do input. O algoritmo executa em tempo constante, independentemente do tamanho dos dados de entrada.

$O(\log n)$: Tempo de execução logarítmico. O tempo aumenta de forma muito lenta à medida que o tamanho da entrada aumenta.

Busca binária em uma lista ordenada. A complexidade cresce de forma logarítmica em relação ao tamanho do input. Cada operação reduz o espaço de busca pela metade.

$O(n)$: Tempo de execução linear. O tempo aumenta diretamente proporcionalmente ao tamanho da entrada.

Exemplo: Percorrer todos os elementos de um array para encontrar um valor. A complexidade cresce linearmente com o tamanho do input. O tempo de execução aumenta proporcionalmente ao número de elementos.

Hierarquia de complexidade

$O(n \log n)$: Tempo de execução quase-linear. Encontrado em muitos algoritmos de ordenação eficientes, como o mergesort e o heapsort. A complexidade cresce de forma intermediária entre linear e quadrática.

$O(n^2)$: Tempo de execução quadrático. O tempo é proporcional ao quadrado do tamanho da entrada. Encontrado em algoritmos de ordenação como o bubblesort e o insertionsort. A complexidade cresce quadraticamente em relação ao tamanho do input. Geralmente ocorre em algoritmos que envolvem loops aninhados.

$O(n^3)$: Tempo de execução cúbico. Encontrado em algoritmos como o algoritmo de multiplicação de matrizes ingênuo. A complexidade cresce cúbicamente em relação ao tamanho do input. Pode ocorrer em algoritmos com três loops aninhados.

Hierarquia de complexidade

$O(2^n)$: Tempo de execução exponencial. O tempo cresce exponencialmente com o tamanho da entrada. Encontrado em algoritmos com crescimento exponencial, como algoritmos de busca completa em espaço de estados.

Exemplo: Algoritmo recursivo para encontrar todas as combinações de um conjunto.

$O(n!)$: Tempo de execução fatorial. O crescimento é muito rápido e torna-se impraticável para entradas maiores.

Exemplo: Algoritmo para encontrar todas as permutações de um conjunto. A complexidade cresce fatorialmente em relação ao tamanho do input. É uma das complexidades mais lentas e deve ser evitada sempre que possível.

Classes de problemas

- Cuidado

- Na análise assintótica as constantes de multiplicação são consideradas irrelevantes e descartadas
 - Porém, elas podem ser relevantes na prática, principalmente se o tamanho da entrada é pequeno
- Exemplo: qual função tem menor custo?
 - $f(n) = 10^{100} * n$
 - $g(n) = 10n \log n$
- Análise assintótica: o primeiro é mais eficiente
 - $f(n) = 10^{100} * n$ tem complexidade $O(n)$
 - $g(n) = 10n \log n$ tem complexidade $O(n \log n)$
- No entanto, 10^{100} é um número muito grande
 - Neste caso, $10n \log n > 10^{100} * n$ apenas para
 - Para qualquer valor menor de n o algoritmo de complexidade $O(n \log n)$ será melhor

$$n > 2^{10^{99}}$$

Análise de Casos de Pior, Médio e Melhor:

Caso de Pior (Worst Case): Este é o cenário em que o algoritmo enfrenta as entradas mais desfavoráveis em termos de tempo ou espaço. A análise do caso de pior fornece um limite superior para o desempenho do algoritmo sob qualquer condição. É útil para garantir que o algoritmo não execute de maneira inaceitavelmente lenta em nenhuma circunstância.

Caso Médio (Average Case): O caso médio envolve calcular o desempenho do algoritmo em uma distribuição de entradas típicas ou realistas. Isso requer considerar as probabilidades de diferentes entradas ocorrerem e calcular a média ponderada do desempenho do algoritmo para essas entradas. A análise do caso médio dá uma ideia mais realista de como o algoritmo se comportará em situações práticas.

Caso Melhor (Best Case): Este é o cenário mais otimista, em que o algoritmo enfrenta as entradas que resultam no melhor desempenho possível. A análise do caso melhor é, em muitos casos, menos útil, pois geralmente não representa uma situação comum. Pode até ser enganosa, já que um algoritmo que funciona excepcionalmente bem em um caso melhor pode ter desempenho insatisfatório em casos reais.

Importância de Considerar Casos Extremos e Médios na Análise:

Garantia de Desempenho: A análise do caso de pior garante que o algoritmo não terá um desempenho inaceitável, mesmo nas piores situações. Isso é crítico para sistemas em que o tempo de execução é um fator crucial, como sistemas de tempo real.

Planejamento de Recursos: A análise do caso médio oferece uma compreensão mais realista do desempenho do algoritmo sob condições normais. Isso ajuda a tomar decisões informadas sobre o dimensionamento de recursos, como processadores, memória e largura de banda.

Avaliação Comparativa: Comparar algoritmos é mais eficaz quando todos são avaliados sob os mesmos critérios. A análise dos três casos permite uma comparação completa dos algoritmos em diferentes cenários.

Otimização Eficiente: A análise do caso de pior ajuda a identificar gargalos e áreas de melhoria. Por outro lado, a análise do caso médio permite otimizar considerando o uso real do algoritmo.

Projeto de Algoritmos: Compreender os três casos permite o desenvolvimento de algoritmos adaptativos que podem ajustar sua abordagem dependendo da natureza das entradas.

Análise de complexidade dos algoritmos: Selection Sort

Entrada: Array A com n elementos

Saída: Array A em ordem crescente

	Linha	melhor caso	pior caso
para i de 0 até n-2 faça	1	$2n+1$	$2n+1$
para j de i+1 até n-1 faça	2	$n^2 + 3n - 4$	$n^2 + 3n - 4$
se $A[j] < A[i]$ então	3	$(3n^2 - 3n)/2$	$(3n^2 - 3n)/2$
temp=A[i]	4	0	$n^2 - n$
$A[i]=A[j]$	5	0	$(3n^2 - 3n)/2$
$A[j]=temp$	6	0	$n^2 - n$
	total	$(5n^2 + 7n - 6)/2$	$6n^2 - 3n$

qualquer caso

Linha 1: para i de 0 até $n-2$ faça

subtração realizada **uma** única vez

atribuição realizada
uma única vez

teste na condição de parada: No comando temos primeiro um teste de condição de parada, se a condição for falsa o laço é executado uma vez, e depois a variável de controle é incrementada. O i varia de 0 até $n-2$, inclusive. A condição então será testada **n vezes**, $n-1$ vezes ela será falsa e na última vez ela será verdadeira.

incremento da variável de controle:

i será incrementado **$n-1$ vezes** (1 menos que o teste de parada, que é o número de vezes que o laço será executado)

Total de operações na linha 1: $2+n+n-1= 2n+1$

qualquer caso

Linha 2: para j de i+1 até n-1 faça

3 operações básicas realizadas uma única vez cada uma delas.

Esta linha é executada n-1 vezes por causa do comando da linha 1, logo **3n-3** operações básicas

Operações realizadas repetidamente:

- teste na condição de parada:

j varia de i+1 até n-1, inclusive. A condição de parada será testada e resultará falsa $n-1-(i+1) + 1$ vezes; e no próximo teste será verdadeira. Então $n-1-i-1 + 1 + 1 = n-i$ para cada valor de i determinado pelo laço externo.

- incremento da variável de controle:

j será incrementada **n-i-1** vezes para cada valor de i determinado pelo laço externo. (1 menos que o teste de parada, que é o número de vezes que o laço será executado)

A medida que i aumenta na linha 1, o teste de condição de parada e o decremento de j são executados mais e mais vezes:

qualquer caso

Linha 2: para j de $i+1$ até $n-1$ faça

3 operações básicas realizadas uma única vez cada uma delas.

Esta linha é executada $n-1$ vezes por causa do comando da linha 1, logo **$3n-3$** operações básicas

Total de operações na linha 2:
 $3n - 3 + (n^2 + n - 2)/2 + (n^2 - n)/2 =$
 $n^2 + 3n - 4$

Ope

Linha 1		Linha 2: j varia		Vezes que será executado...		
Iteração do para	i	de $i+1$	até $n-1$	teste da condição de parada na linha 2: $n-i$	decremento de j na linha 2: $n-i-1$	comando se na linha 3
1ª	0	1	$n-1$	$n-0 = n$	$n-0-1 = n-1$	$n-1$
2ª	1	2	$n-1$	$n-1$	$n-1-1 = n-2$	$n-2$
3ª	2	3	$n-1$	$n-2$	$n-2-1 = n-3$	$n-3$
...	...	...	...	...	...	...
$n-2$	$n-3$	$n-1$	$n-1$	$n-(n-3) = 3$	$n-(n-3)-1 = 2$	2
$n-1$	$n-2$	n	$n-1$	$n-(n-2) = 2$	$n-(n-2)-1 = 1$	1
Total				$n(n+1)/2 - 1 = (n^2 + n - 2)/2$	$n(n-1)/2 = (n^2 - n)/2$	$(n^2 - n)/2$

A medida que i aumenta na linha 1, o teste de condição de parada e o decremento de j são executados mais e mais vezes:

qualquer caso

Linha 3: se $A[j] < A[i]$

3 operações básicas realizadas uma única vez cada uma delas.

A medida que i aumenta na linha 1, o teste de condição de parada e o decremento de j são executados mais e mais vezes:

Linha 1		Linha 2: j varia		Vezez que será executado...		
Iteração do para	i	de $i + 1$	até $n - 1$	teste da condição de parada na linha 2: $n - i$	decremento de j na linha 2: $n - i - 1$	comando se na linha 3
1ª	0	1	$n - 1$	$n - 0 = n$	$n - 0 - 1 = n - 1$	$n - 1$
2ª	1	2	$n - 1$	$n - 1$	$n - 1 - 1 = n - 2$	$n - 2$
3ª	2	3	$n - 1$	$n - 2$	$n - 2 - 1 = n - 3$	$n - 3$
...	...	...	...	...	...	...
$n - 2$	$n - 3$	$n - 1$	$n - 1$	$n - (n - 3) = 3$	$n - (n - 3) - 1 = 2$	2
$n - 1$	$n - 2$	n	$n - 1$	$n - (n - 2) = 2$	$n - (n - 2) - 1 = 1$	1
Total				$n(n + 1)/2 - 1 = (n^2 + n - 2)/2$	$n(n - 1)/2 = (n^2 - n)/2$	$(n^2 - n)/2$

Total de operações na linha 2:
 $3 \cdot (n^2 - n)/2 = (3n^2 - 3n)/2$

melhor caso

Linha 4 a 6:

```
temp=A[i]  
A[i]=A[j]  
A[j]=temp
```

No melhor caso o array já está ordenado o comando da linha 3 será sempre falso e estes comando internos não serão executados.

Total de operações nas linha 4, 5 e 6: **0**

Linha 4 a 6:

`temp=A[i]` → 2 operações: 1 indexação e 1 atribuição
`A[i]=A[j]` → 3 operações: 2 indexações e 1 atribuição
`A[j]=temp` → 2 operações: 1 indexação e 1 atribuição

No pior caso os comando dentro do se serão executados $(n^2-n)/2$ vezes

Total de operações nas linha

4: n^2-n

5: $(3n^2-3n)/2$

6: n^2-n

Análise resumida

Linha		Operações no pior caso			Repetições do laço
N.º	Código	Tipo	Descrição	Quantidade	
1	para i de 0 até n-2 faça	Aritméticas	n-2	1	n - 1
		Inicialização	i=0	1	
		Incrementos	i++	n - 1	
		Comparações	i <= n-2	n	
		Total linha 1:			2n + 1
2	para j de i+1 até n-1 faça	Aritméticas	i+1, n-1	2	n - 2 $\sum_{i=0}^{n-2} (n-i-1) = \frac{1}{2}(n^2 - n)$
		Inicialização	j=i+1	1	
		Incrementos	j++	n - i - 1	
		Comparações	j <= n-1	n - i	
		Total linha 2:			$\sum_{i=0}^{n-2} 2(n-i-1) = n^2 + 3n - 4$
3	se A[j] < A[i] então	Indexações	A[]	1	
		Comparações	<	1	
		Total linha 3:			3(n² - n)/2
4	temp = A[i]	Indexações	A[]	1	
		Atribuições	=	1	
		Total linha 4:			2 × (n² - n)/2 = n² - n
5	A[i] = A[j]	Indexações	A[]	2	
		Atribuições	=	1	
		Total linha 5:			3(n² - n)/2
6	A[j] = temp	Indexações	A[]	1	
		Atribuições	=	1	
		Total linha 6:			2 × (n² - n)/2 = n² - n

Bubble sort

Algoritmo: *Bubble sort* (A, n)

Entrada: Array A com n elementos

Saída: Array A em ordem crescente

- 1) para i de 0 até $n-1$ passo 1 faça
- 2) para j de $n-1$ até $i+1$ passo -1 faça
- 3) se $A[j-1] > A[j]$ então
- 4) $\text{temp} = A[j-1]$
- 5) $A[j-1] = A[j]$
- 6) $A[j] = \text{temp}$

NOTA: A indentação mostra o escopo dos comandos:
o para na linha 2 está dentro do para na linha 1
o se na linha 3 está dentro do para na linha 2
as linhas 4 a 6 estão dentro do se na linha 3

Linha	Melhor caso	Pior caso
1	$2n + 3$	$2n + 3$
2	$n^2 + 3n$	$n^2 + 3n$
3	$2n^2 - 2n$	$2n^2 - 2n$
4	0	$(3n^2 - 3n)/2$
5	0	$2n^2 - 2n$
6	0	$n^2 - n$
Total	$3n^2 + 3n + 3$	$(15n^2 - 3n + 6)/2$

Algoritmo: Insertion sort (A, n)

Entrada: Array A com n elementos

Saída: Array A em ordem crescente

- 1) para i de 1 até $n-1$ faça
- 2) atual = $A[i]$
- 3) $j = i-1$
- 4) enquanto $j \geq 0$ e $A[j] > \text{atual}$ faça
- 5) $A[j+1] = A[j]$
- 6) $j = j-1$
- 7) $A[j+1] = \text{atual}$

NOTA: A indentação mostra o escopo dos comandos:

linhas 2 a 7: dentro do para na linha 1

linhas 5 e 6: dentro do enquanto na linha 4

Linha	Melhor caso	Pior caso
1	$2n + 1$	$2n + 1$
2	$2n - 2$	$2n - 2$
3	$2n - 2$	$2n - 2$
4	$4n - 4$	$2n^2 + 2n - 4$
5	0	$2n^2 + 2n$
6	0	$n^2 - n$
7	$3n - 3$	$3n - 3$
Total	$13n - 10$	$5n^2 + 8n - 10$

	Linha	Melhor caso	Pior caso
Algoritmo: Transpor (M, n) Entrada: Matriz M com n linhas e n colunas Saída: Matriz M transposta 1) para i de 1 até $n-1$ faça 2) para j de 0 até $i-1$ faça 3) se $M[i,j] \neq M[j,i]$ 4) $\text{temp} = M[i,j]$ 5) $M[i,j] = M[j,i]$ 6) $M[j,i] = \text{temp}$ NOTA: A indentação mostra o escopo dos comandos: <i>para na linha 2: dentro do para na linha 1</i> <i>se na linha 3: dentro do para na linha 2</i> <i>linhas 4 a 6: dentro do se na linha 3</i>	1	$2n + 1$	$2n + 1$
	2	$n^2 + 2n - 3$	$n^2 + 2n - 3$
	3	$3(n^2 - n)/2$	$3(n^2 - n)/2$
	4	0	$n^2 - n$
	5	0	$3(n^2 - n)/2$
	6	0	$n^2 - n$
	Total	$(5n^2 + 5n - 4)/2$	$6n^2 - n - 2$

	Linha	Melhor caso	Pior caso
Algoritmo: Girar (M, n) Entrada: Matriz M com n linhas e n colunas; n é par Saída: Matriz M girada 90° no sentido horário 1) para i de 0 até $(n/2)-1$ faça 2) para j de i até $n-2-i$ faça 3) temp = $M[n-1-j, i]$ 4) $M[n-1-j, i]$ = $M[n-1-i, n-1-j]$ 5) $M[n-1-i, n-1-j]$ = $M[j, n-1-i]$ 6) $M[j, n-1-i]$ = $M[i, j]$ 7) $M[i, j]$ = temp NOTA: A indentação mostra o escopo dos comandos: para na linha 2: dentro do para na linha 1 linhas 3 a 7: dentro do para na linha 2	1	$n + 4$	$n + 4$
	2	$(n^2 + 4n)/2$	$(n^2 + 4n)/2$
	3	$4n^2/4$	$4n^2/4$
	4	$9n^2/4$	$9n^2/4$
	5	$9n^2/4$	$9n^2/4$
	6	$5n^2/4$	$5n^2/4$
	7	$2n^2/4$	$2n^2/4$
	Total	$(31n^2 + 12n + 16)/4$	$(31n^2 + 12n + 16)/4$

stupid sort

Algoritmo ordenando de forma crescente.

Lembrando que as entradas estão:

caso 1: em ordem crescente (já ordenado)

caso 2: em ordem decrescente

	Linha	Melhor caso	Pior caso
Algoritmo: Stupid sort (A, n)			
Entrada: Array A com n inteiros não negativos			
Saída: Array A em ordem crescente			
1) para iB de 0 até n excl faça	1	$2n + 2$	$2n + 2$
2) para iA de 0 até n excl faça	2	$2n$	$n^2 + n$
3) se $A[iA] \neq -1$ então	3	$2n$	$n^2 + n$
4) menor = $A[iA]$	4	$2n$	$2n$
5) saia do para na linha 2	5	n	n
6) para iA de 0 até n excl faça	6	$2n^2 + 2n$	$2n^2 + 2n$
7) se $A[iA] \neq -1$ e $A[iA] < \text{menor}$ então	7	$5n^2$	$5n^2$
8) menor = $A[iA]$	8	$n^2 - n$	0
9) $B[iB] = \text{menor}$	9	$2n$	$2n$
10) para iA de 0 até n excl faça	10	$n^2 + n$	$n^2 + n$
11) se $A[iA] = \text{menor}$ então	11	$n^2 + n$	$n^2 + n$
12) $A[iA] = -1$	12	$2n$	$2n$
13) saia do para na linha 10	13	n	n
14) para iA de 0 até n excl faça	14	$2n + 2$	$2n + 2$
15) $A[iA] = B[iA]$	15	$3n$	$3n$
NOTA 1: A indentação mostra o escopo dos comandos: o para na linha 2 está dentro do para na linha 1 o se na linha 3 está dentro do para na linha 2 as linhas 4 e 5 estão dentro do se na linha 3	Total	$10n^2 + 22n + 4$	$11n^2 + 21n + 4$
NOTA 2: a semântica do para...excl é igual à de C: para iB de 0 até n excl faça equivale a for ($iB=0$; $iB<n$; $iB++$)			